

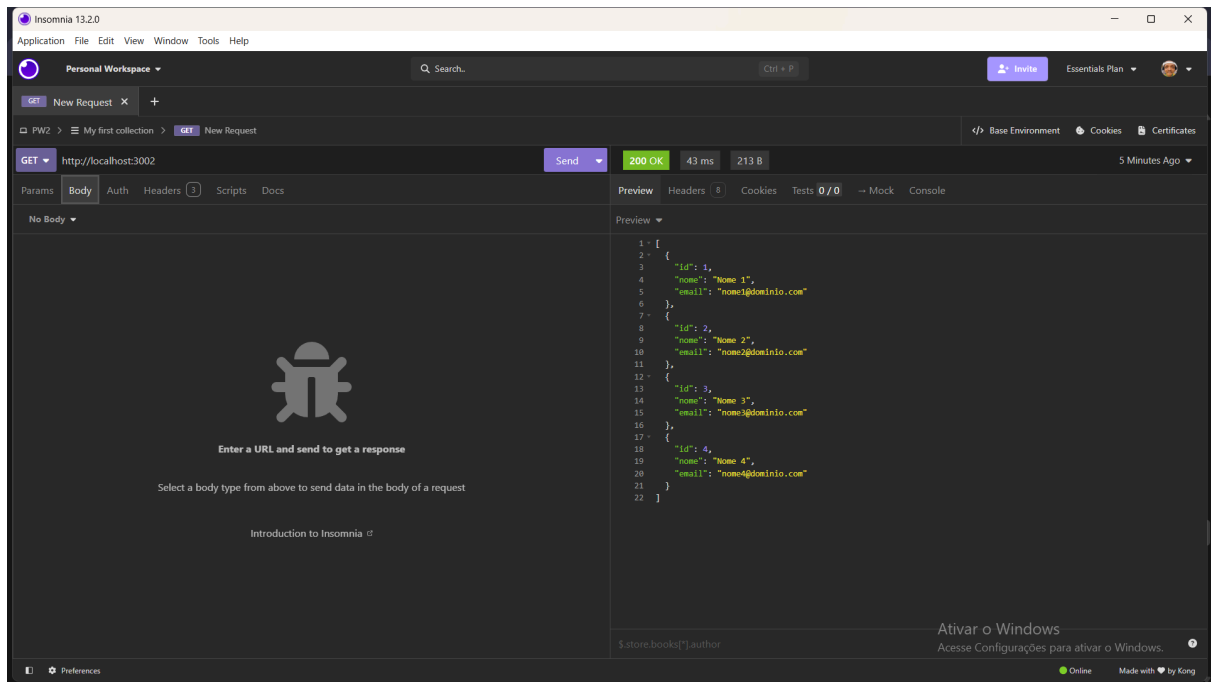
API REST com Node.js e MySQL: CRUD Completo Sem ORM

Nome: Maria Eduarda de Páscoa Carneiro

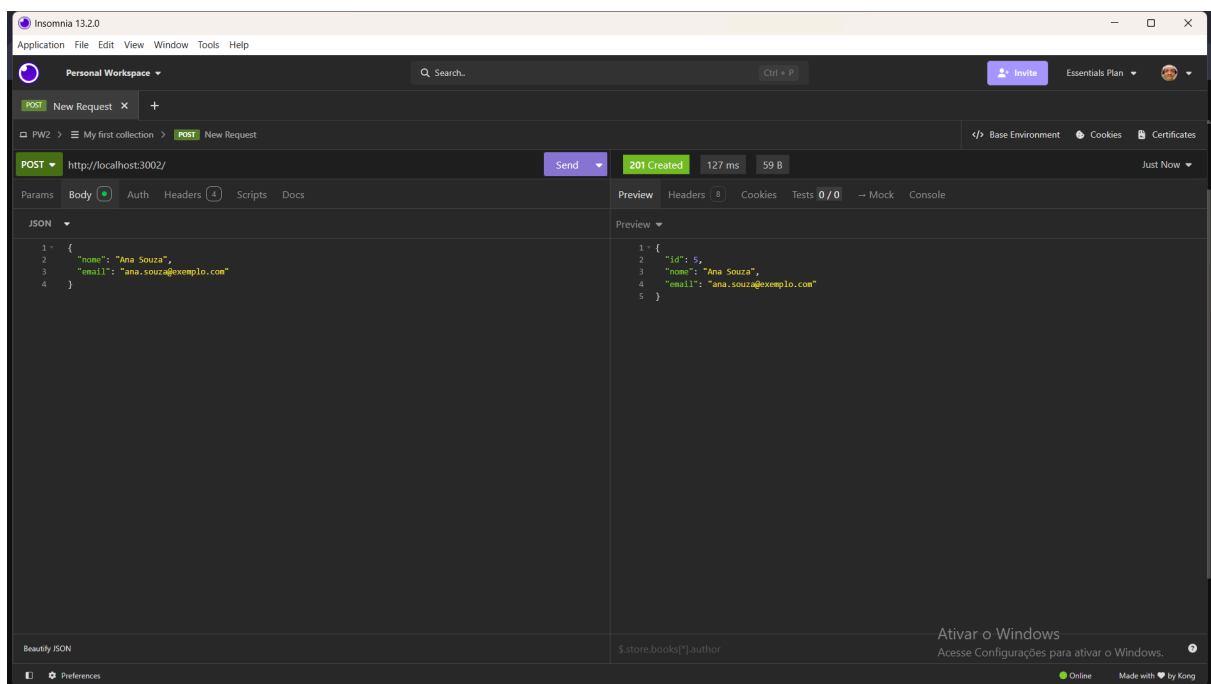
Série: 2ºDS Manhã

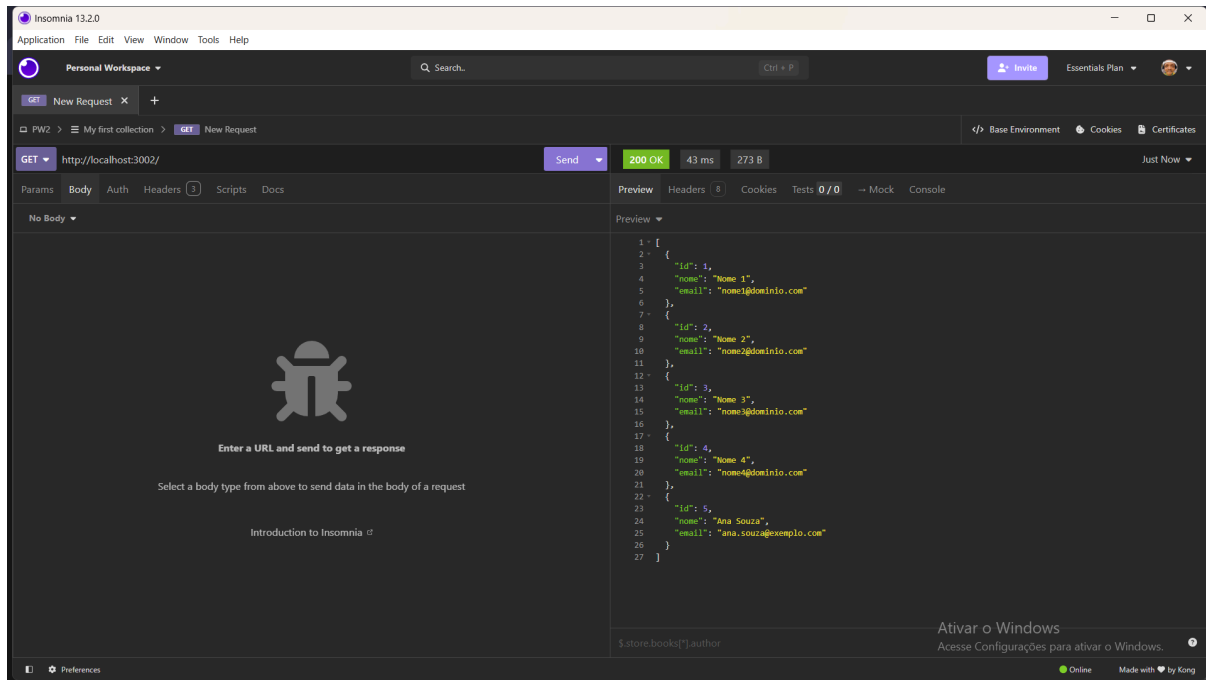
RM: 20250496

1. Teste 1 — Listar usuários (READ)

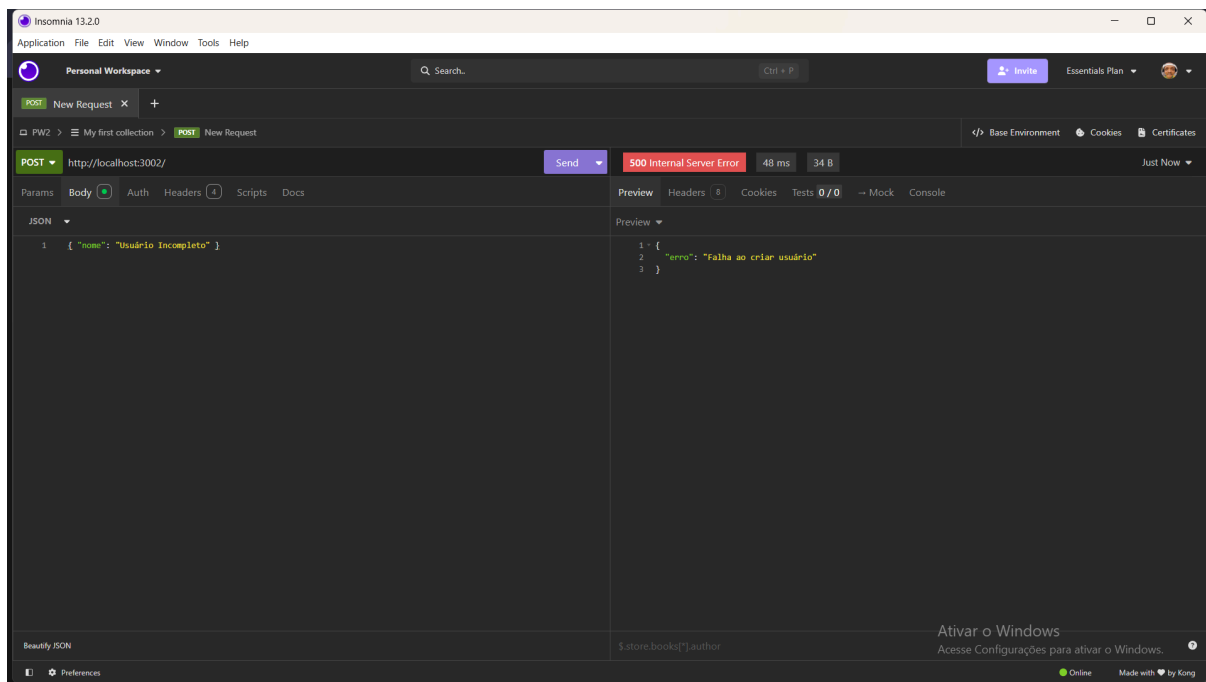


2. Teste 2 — Criar um usuário válido (CREATE)





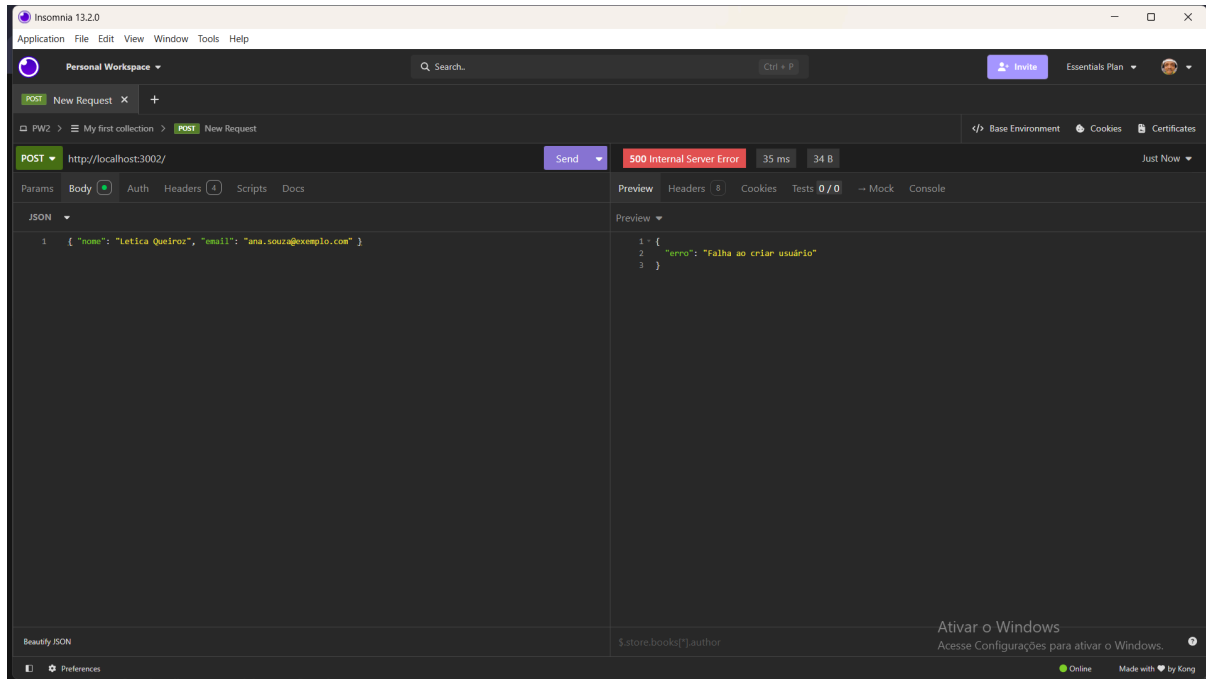
3. Teste 3 — Criar um usuário sem o campo email (validação)



- **Pergunta para reflexão:** um status 500 genérico é uma boa resposta para esse caso? Que status seria mais adequado (dica: 400 Bad Request) e o que precisaria mudar no código para diferenciar "erro de validação" de "erro interno do servidor"?

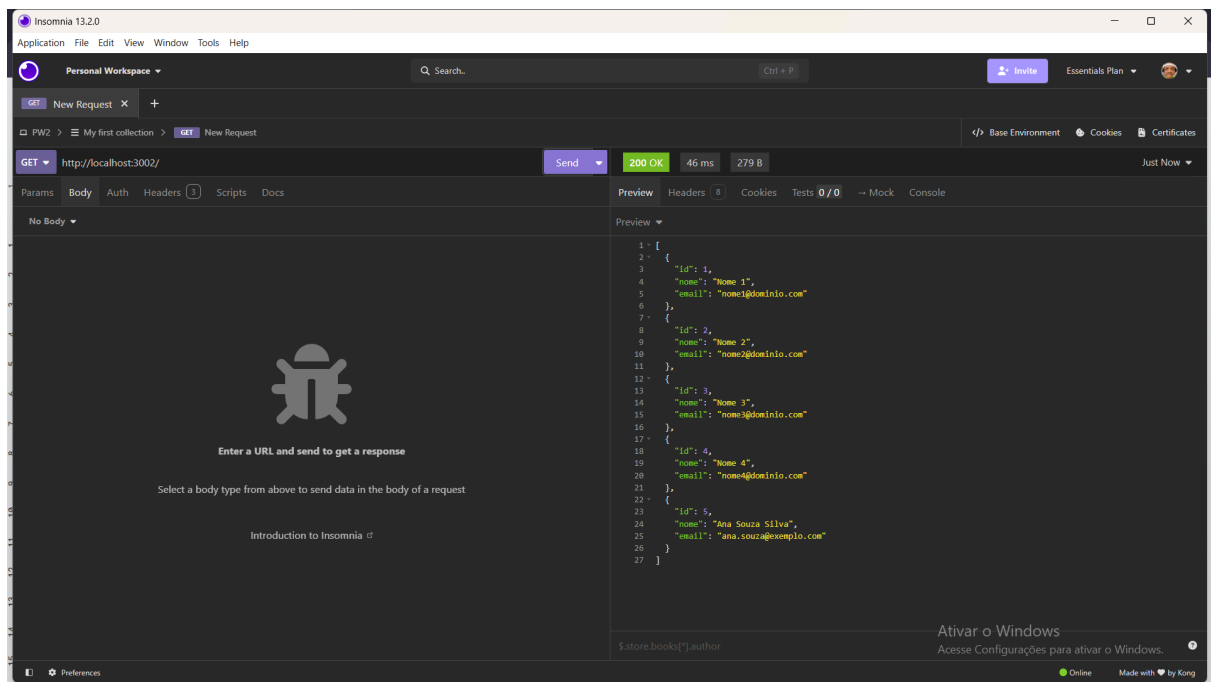
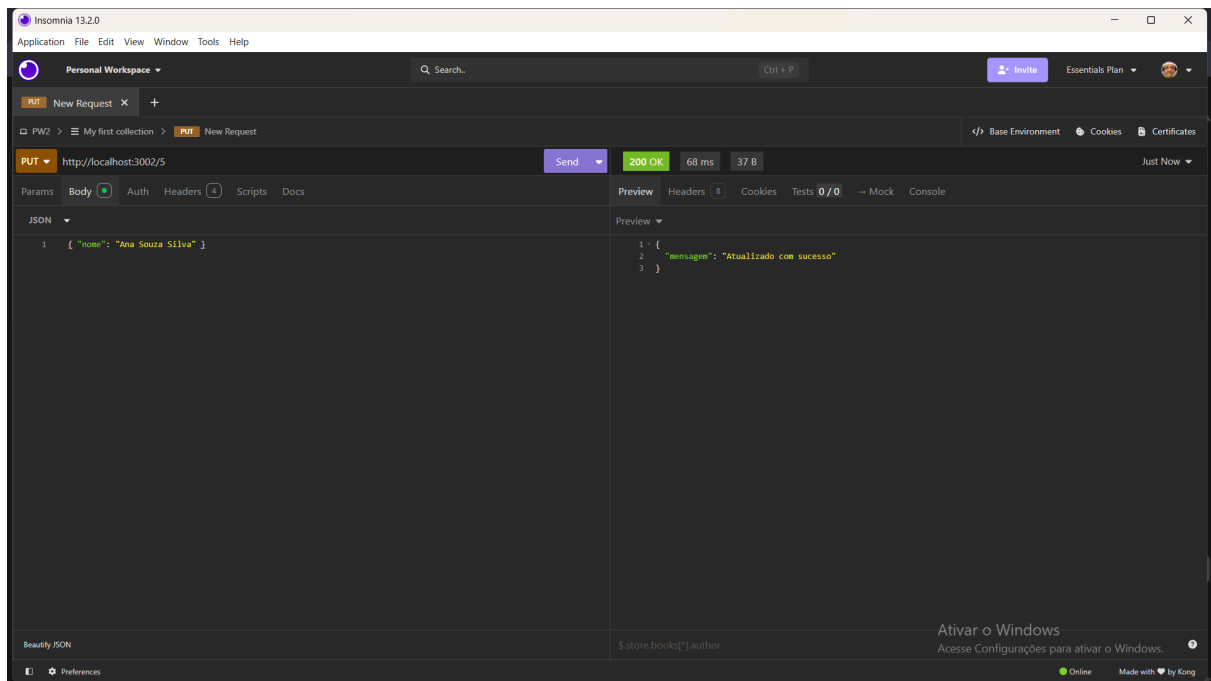
- **Resposta:** Não, pois o status 500 é feito para tratar erros internos que quebram o servidor, nesse caso o erro é de que faltam informações obrigatórias na requisição (email), portanto o status mais adequado seria o 400 (Bad Request), pois ele é feito para lidar com requisições mal construídas/enviadas.

4. Teste 4 — Criar um usuário com e-mail duplicado

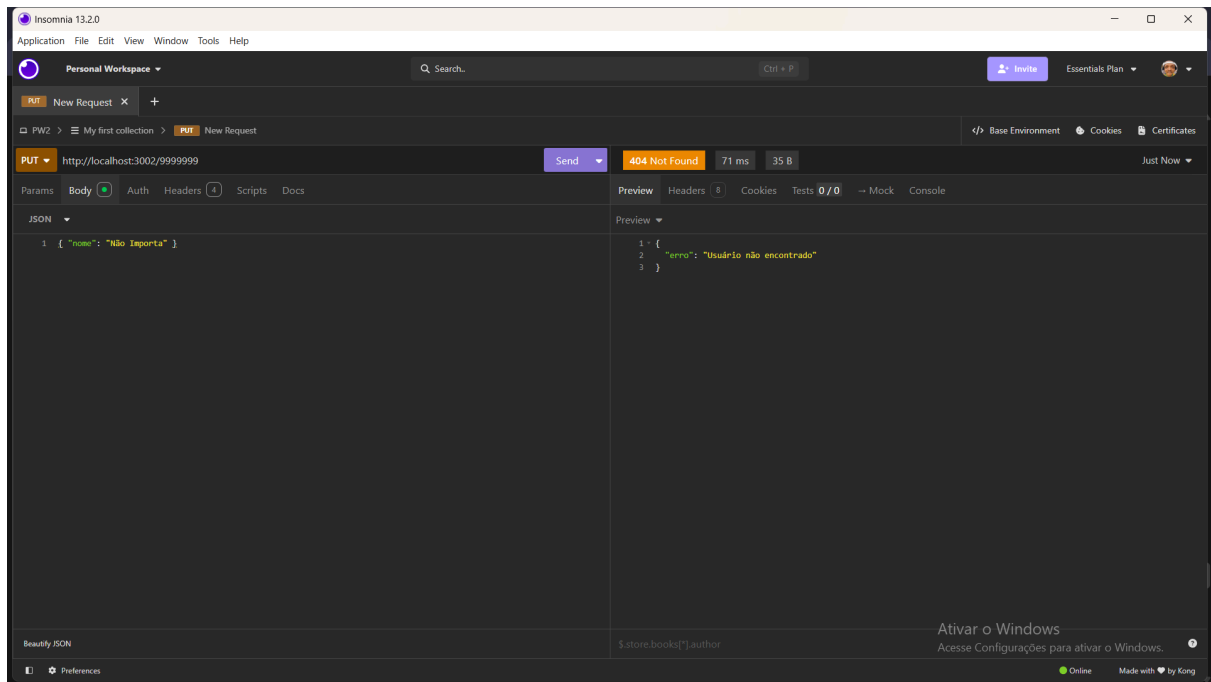


- **Pergunta para reflexão:** Como você trataria esse caso de forma mais específica (ex: 409 Conflict)?
- **Resposta:** Eu mudaria o status de 500 para 409 (Conflict), pois mesmo que o corpo da requisição enviado esteja correto, o servidor não roda a requisição pois o campo email é UNIQUE e já existe alguém com esse email no banco. Essa troca de status faz com que o usuário entenda melhor que o erro foi por causa de um conflito entre os dados e não por algo interno do servidor.

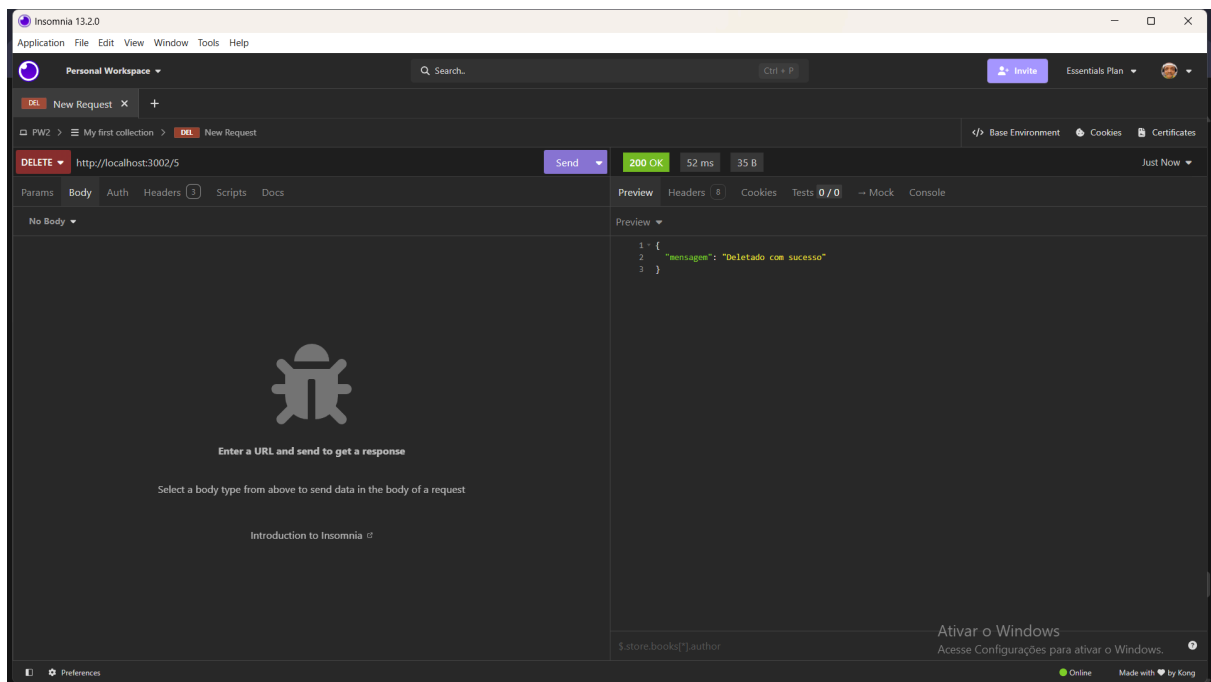
5. Teste 5 — Atualizar um usuário existente (UPDATE)

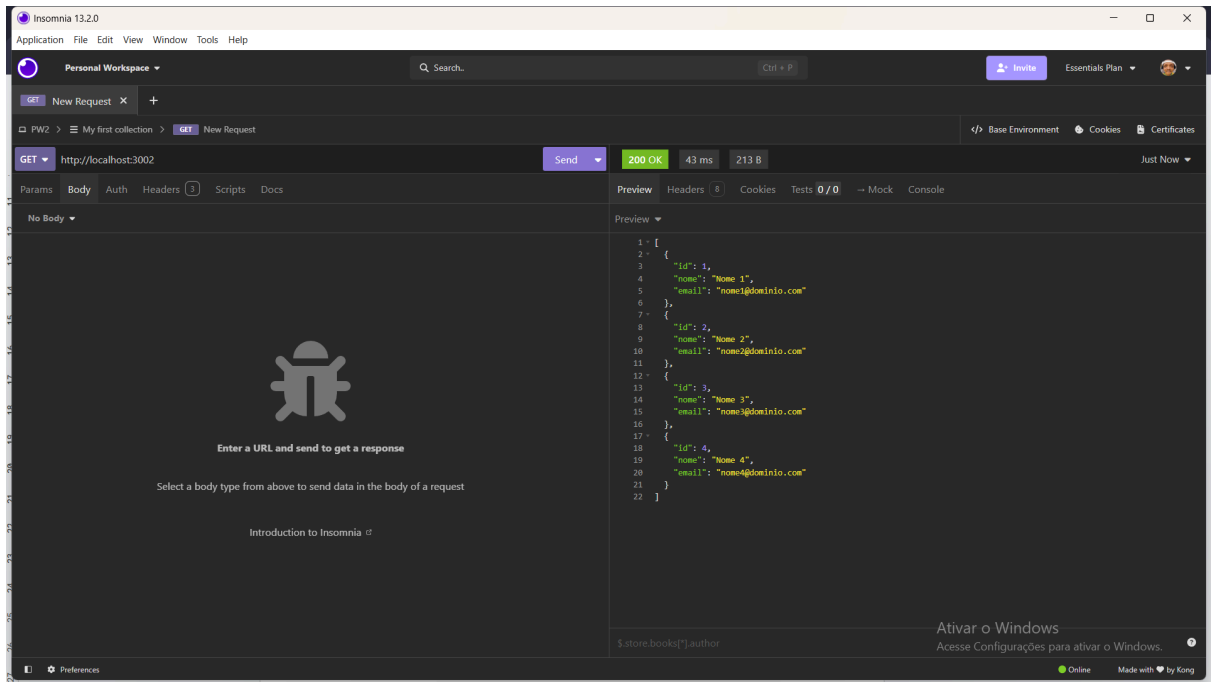


6. Teste 6 — Atualizar um usuário que não existe



7. Teste 7 — Remover um usuário existente (DELETE)





8. Teste 8 — Remover um usuário que não existe

